

Git introduction

1. What is Git?

Git is a distributed version control system used to track code changes.

What is Git Bash?

Git Bash is a command-line terminal for Windows that allows users to run Git commands and Linux-style commands easily.

2. Why do developers use Git?

Developers use Git to manage code, track changes, and work in teams.

3. What is a Git repository?

A Git repository is a storage place where project files and history are saved.

4. What is the difference between Git and GitHub?

Git is a version control tool, while GitHub is an online platform for Git projects.

5. How do you initialize a Git repository?

You can initialize a Git repository using the command below.

```
git init
```

6. What is a commit in Git?

A commit is a saved snapshot of project changes in Git.

7. What is the purpose of the `git clone` command?

The `git clone` command is used to copy a remote repository to a local machine.

```
git clone repository-link
```

8. What is a branch in Git?

A branch is a separate workspace in Git used to develop features independently.

9. What does the `git pull` command do?

The `git pull` command downloads and updates the latest changes from a remote repository.

```
git pull
```

10. Why is Git important for teamwork?

Git helps teams collaborate, manage code safely, and avoid conflicts.

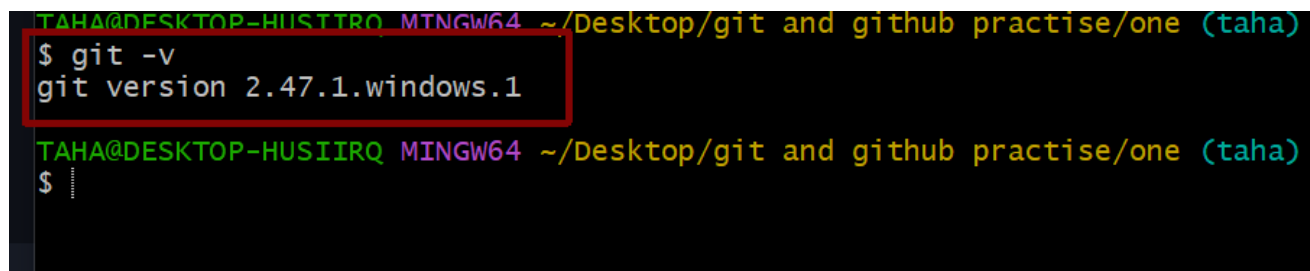
Git start

How to Check Git Version

Git version command is used to check Git is installed and to display the current installed version of Git.

```
git -v
```

```
git version 2.49.0
```

A terminal window screenshot with a black background and colorful text. The prompt is 'TAHA@DESKTOP-HUSTTRO MINGW64 ~/Desktop/git and github practise/one (taha)'. The command '\$ git -v' is entered, and the output 'git version 2.47.1.windows.1' is displayed. A red rectangular box highlights the command and its output. Below this, the prompt changes to 'TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)' and a new '\$' prompt is visible.

```
TAHA@DESKTOP-HUSTTRO MINGW64 ~/Desktop/git and github practise/one (taha)
$ git -v
git version 2.47.1.windows.1
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
```

How to Set Git Username

Git username command is used to set the name that will appear in commit history for identification.

```
git config --global user.name "Your Name"
```

```
git config --global user.name "Muhammad Taha"
```

```
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.name "taha" 1
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.email "muhammادتaha00224443@GMAIL.COM" 2
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.email
muhammادتaha00224443@GMAIL.COM 3
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.name
taha 4
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
```

How to Set Git Email

Git email command is used to set the email address associated with every Git commit.

```
git config --global user.email "your@email.com"
```

```
git config --global user.email "taha@gmail.com"
```

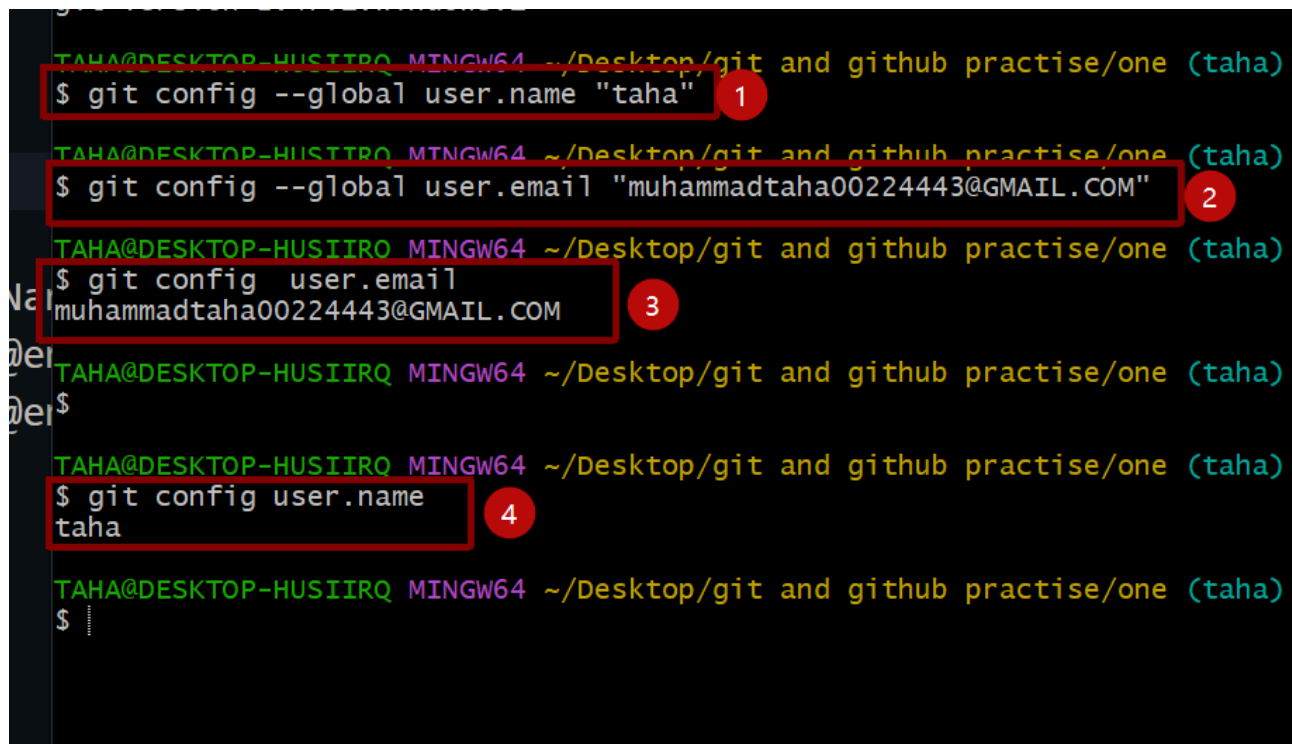
```
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.name "taha" 1
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.email "muhammادتaha00224443@GMAIL.COM" 2
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.email
muhammادتaha00224443@GMAIL.COM 3
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.name
taha 4
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
```

How to Check Git Username

This command is used to display the currently configured Git username.

```
git config user.name
```

```
Muhammad Taha
```



A terminal window showing the configuration of Git username and email. The prompt is 'TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)'. The steps are: 1. '\$ git config --global user.name "taha"', 2. '\$ git config --global user.email "muhammادتaha00224443@GMAIL.COM"', 3. '\$ git config user.email muhammادتaha00224443@GMAIL.COM', and 4. '\$ git config user.name taha'. Each command is highlighted with a red box and a red circle containing the step number.

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.name "taha"
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.email "muhammادتaha00224443@GMAIL.COM"
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.email
muhammادتaha00224443@GMAIL.COM
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.name
taha
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
```

How to Check Git Email

This command is used to display the currently configured Git email address.

```
git config user.email
```

```
taha@gmail.com
```

```
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.name "taha" 1
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config --global user.email "muhammادتaha00224443@GMAIL.COM" 2
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.email
muhammادتaha00224443@GMAIL.COM 3
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$ git config user.name
taha 4
TAHA@DESKTOP-HUSTIIRQ MINGW64 ~/Desktop/git and github practise/one (taha)
$
```

How to Create a Local Git Repository

```
git init
```

- `git init` means Git initialization.
- This command creates a new local Git repository.
- Git creates a hidden `.git` folder in the current directory.
- A file or folder starting with `.` is called a hidden file or folder in Linux.
- Hidden files can be viewed using:

```
ls -a
```

```
ls -al
```

- Files exist in the folder, but Git does not track them automatically.

- Git cannot track files until they are added using Git commands.

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5
$ git init
Initialized empty Git repository in C:/Users/TAHA/Desktop/git and github practise/test5/.git/

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ ls -a
./ ../ .git/

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ ls -al
total 8
drwxr-xr-x 1 TAHA 197121 0 May  9 23:11 ./
drwxr-xr-x 1 TAHA 197121 0 May  9 23:11 ../
drwxr-xr-x 1 TAHA 197121 0 May  9 23:11 .git/

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$
```

-

Git Status

```
git status
```

- This command shows the current status of files in the Git repository.
- It tells whether Git is tracking the files or not.
- **Untracked files** are files that Git does not know about yet.
- **Red color** means Git is not tracking the file.
- **Green color** means the file is staged and ready for commit.

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   file1

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    file2

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$
```

-

Git Add

```
git add filename
```

- `git add` command moves files from the untracked area to the staging area.
- This command adds files to the staging area, not to GitHub.
- After using `git add`, Git becomes aware of the file.
- When you run `git status`, the file appears in green color.
- Green color means the file is staged and ready for commit.
- you can add multiple file using
 - `git add file1 file 2 file 3`
- for all file in the staging area using
 - `git add .`
 - `.` mean all file

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git add .

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   file1
    new file:   file2

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$
```

Git Commit

```
git commit -m "message"
```

- `git commit` command is used to save changes permanently in the local Git repository.
- A commit creates a snapshot of the staged files.
- `-m` is used to add a short commit message.

- Commit messages help identify what changes were made.

```
TAHA@DESKTOP-HUSTIRO MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git commit -m "first commit"
[main (root-commit) 862bf8a] first commit
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file1
create mode 100644 file2

TAHA@DESKTOP-HUSTIRO MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git log
commit 862bf8a1ce25b4f2283ac6e22da200cab99441dd (HEAD -> main)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sat May 9 23:17:13 2026 +0500

    First commit

TAHA@DESKTOP-HUSTIRO MINGW64 ~/Desktop/git and github practise/test5 (main)
$
```

Git Log

```
git log
```

- `git log` command is used to view commit history in Git.
- It shows all previous commits with details.
- You can check commit messages, commit ID, author, and date.
- Red color** means the file is not in the staging area.
- Green color** means the file is in the staging area.

```
TAHA@DESKTOP-HUSTIRO MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git commit -m "first commit"
[main (root-commit) 862bf8a] first commit
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file1
create mode 100644 file2

TAHA@DESKTOP-HUSTIRO MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git log
commit 862bf8a1ce25b4f2283ac6e22da200cab99441dd (HEAD -> main)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sat May 9 23:17:13 2026 +0500

    First commit

TAHA@DESKTOP-HUSTIRO MINGW64 ~/Desktop/git and github practise/test5 (main)
$
```

Git Commit with Message

```
git commit -m "this is 2nd commit"
```

- This command allows you to write a commit message without opening the Vim editor.

- `-m` stands for message.
- The message describes the changes made in the commit.

Default branch

- Old Git ma default branch "master" hoti thi
- New Git versions ma mostly "main" hoti ha

Delete local repository

`rm -rf .git`

- is sa sirf Git repository delete hoti ha
- project files delete nai hoti

Custom branch name

`git init --initial-branch=main`

- new repo "main" branch ke sath create hoti ha

Git status verbose

`git status -v`

- `v` mean verbose/detail

Git status short

`git status -s`

- short output show krta ha

??

?? filename

- mean Git abhi file ko track nai kr raha
- untracked file

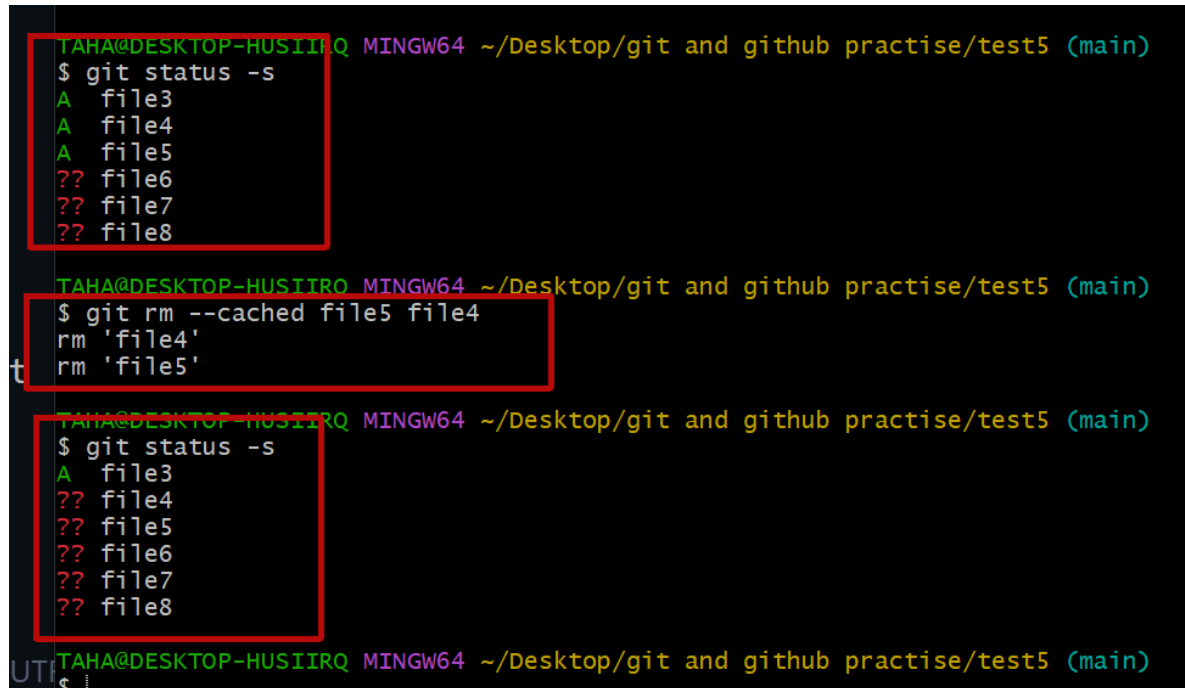


```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
it $ git status -s
A file3
A file4
A file5
?? file6
?? file7
?? file8
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$
```

How to Undo a Staged File

```
git rm --cached f4 f5
```

- This command removes files from the staging area.
- It is used when files are added to staging by mistake.
- The files are removed from staging but remain in the local folder.

A terminal window screenshot showing the process of removing staged files. The prompt is 'TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)'. The first command is '\$ git status -s', which shows a list of files: 'A file3', 'A file4', 'A file5', '?? file6', '?? file7', and '?? file8'. The second command is '\$ git rm --cached file5 file4', which outputs 'rm 'file4'' and 'rm 'file5''. The third command is '\$ git status -s', which shows the updated list: 'A file3', '?? file4', '?? file5', '?? file6', '?? file7', and '?? file8'. The prompt is 'TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)'.

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git status -s
A  file3
A  file4
A  file5
?? file6
?? file7
?? file8

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git rm --cached file5 file4
rm 'file4'
rm 'file5'

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git status -s
A  file3
?? file4
?? file5
?? file6
?? file7
?? file8

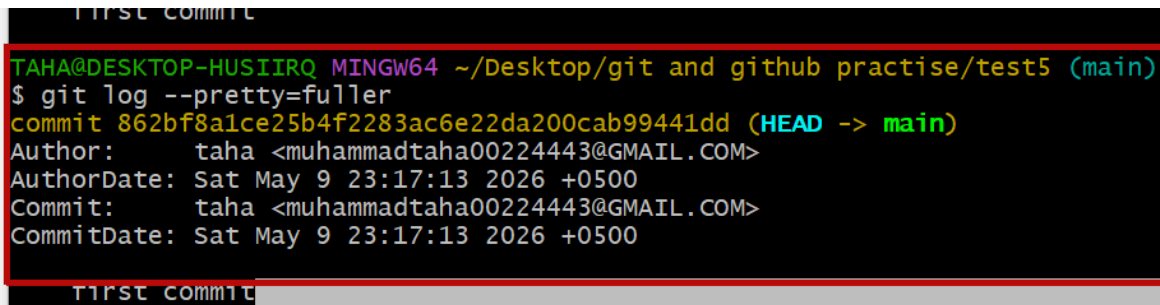
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
```

Git Commit Information

```
git commit -m "this is first commit"
```

- A commit saves staged changes permanently in the local repository.
- Every commit contains 4 main pieces of information:
 - Who created the commit
 - When the commit was submitted
 - Commit message
 - Snapshot of files in the staging area
- A snapshot means Git captures the complete state of the files.
- Git stores snapshots using SHA-1 (Secure Hashing Algorithm 1).
- SHA-1 generates a unique ID for each commit.
- Git creates the first branch automatically after the first commit.

- **HEAD** is basically a pointer that points to the current branch or latest commit.

A terminal window screenshot showing the output of the command 'git log --pretty=fuller'. The output shows a single commit with hash '862bf8a1ce25b4f2283ac6e22da200cab99441dd' and the message '(HEAD -> main)'. The commit details include the author 'taha <muhammادتaha00224443@GMAIL.COM>' and the date 'Sat May 9 23:17:13 2026 +0500'. The terminal text is as follows:

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/test5 (main)
$ git log --pretty=fuller
commit 862bf8a1ce25b4f2283ac6e22da200cab99441dd (HEAD -> main)
Author: taha <muhammادتaha00224443@GMAIL.COM>
AuthorDate: Sat May 9 23:17:13 2026 +0500
Commit: taha <muhammادتaha00224443@GMAIL.COM>
CommitDate: Sat May 9 23:17:13 2026 +0500
```

Git Commit Location

- All Git commits are stored inside the **.git/objects** directory.
- Git saves commit data, snapshots, and other objects in this folder.

Git Modify

- If you change an existing tracked file and run **git status**, Git shows it as a modified file.
- Modified means Git is already tracking the file and detects new changes in it.

Git Commit with **-a** Option

```
git commit -a -m "comment pass"
```

- This command is used to commit modified tracked files directly.
- With **-a**, there is no need to run **git add** for already tracked files.
- It automatically stages tracked modified files and creates the commit.

How to Modify the Last Commit

```
git commit --amend
```

- This command is used to modify the last commit.
- It opens the Vim editor to update the previous commit message.
- You can also add new staged changes to the same commit.

How to Modify the Last Commit Without Changing the

Message

```
git commit --amend --no-edit
```

- This command modifies the last commit without changing the previous commit message.
 - It is used when you want to add new staged files to the last commit.
-

Git Commit with Sign-off

```
git commit -s -m "task done"
```

- `-s` adds a Signed-off-by line in the commit message.
 - It shows who approved or created the commit.
 - `-m` is used to write the commit message directly from the command line.
-

How to Create a Dummy Empty Commit

```
git commit --allow-empty -m "dummy"
```

- This command creates an empty commit without any file changes.
 - It is mainly used for testing, pipelines, or triggering workflows.
-

What is Git Log

- `git log` command is used to view commit history in Git.
-

Show Last 2 Commits

```
git log -n 2
```

- Shows only the last 2 commits.
-

Show Last 3 Commits

```
git log -n 3
```

- Shows only the last 3 commits.
-

Show Short Commit Details

```
git log --pretty=short
```

- Displays commit information in short format.
-

Show Full Commit Details

```
git log --pretty=full
```

- Displays full commit details with author information.
-

Show Full Details with Dates

```
git log --pretty=fuller
```

- Displays complete commit details including dates.
-

Show Commit with File Changes

```
git log -p
```

- Shows commit history with file changes and differences.
-

Show Commit ID and Message in One Line

```
git log --pretty=oneline
```

- Displays each commit in a single line.
-

Show Only Short Commit Hash

```
git log --pretty=format:"%h"
```

- Displays only the short commit ID.
-

Show Commit Hash and Message

```
git log --pretty=format:"%h %s"
```

- Displays commit ID with commit message.
-

Show Commit Hash and Author Name

```
git log --pretty=format:"%h %an"
```

- Displays commit ID with author name.
-

Show Commit Hash and Author Email

```
git log --pretty=format:"%h %ae"
```

- Displays commit ID with author email.
-

Check Last Week Commits

```
git log --since="1 week ago"
```

- Shows commits from the last week.

```
git log --since="yesterday"
```

- Shows commits from yesterday.

```
git log --since="2026-05-07" --until="2026-05-15"
```

- Shows commits between specific dates.

Search Commits by Author

```
git log --author="taha"
```

- Displays commits created by a specific author.

Search Commits by Commit Message

```
git log --grep="first commit"
```

- Searches commits using commit messages.

Git Reset

- `git reset` command is used to undo commits in Git.
- It can discard commits and move changes between staging and working areas.
- Git reset has 3 main modes:
 - Soft

- Mixed
- Hard

How to Undo a Commit Using Soft Reset

- First check the commit ID using:

```
git log
```

```
git reset --soft 339a9cfba62625ff1f22e44575390442622b5d0
```

- This command removes the last commit but keeps all changes in the staging area.
- File changes will still exist.
- Git moves the commit changes back to staging.

```
git diff --staged
```

- This command shows staged changes after reset.

How to Undo a Commit Using Mixed Reset

```
git reset --mixed 339a9cfba62625ff1f22e44575390442622b5d0
```

- This command removes the commit and moves changes to the working directory.
- Changes are removed from the staging area.
- Files become unstaged.

How to Undo a Commit Using Hard Reset

```
git reset --hard 339a9cfba62625ff1f22e44575390442622b5d0
```

- This command removes the commit and deletes all changes permanently.
- Files return to their previous state.
- Both staging area and working directory changes are lost n.

Git Revert

- `git reset --hard` is a dangerous command because it permanently removes changes and can create problems in production.
- `git revert` is a safer command used to undo changes without deleting commit history.
- `git revert` creates a new commit with revert changes, so history remains visible.
- In companies and production environments, developers mostly use `git revert`.
- `git reset --hard` is mostly used in personal projects or personal branches.

```
git revert 4e74a60
```

What does Git Checkout do?

- `git checkout` is used to switch branches or restore files.

```
git checkout branch-name
```

1. Non-Staging Area

- The non-staging area contains modified files that are not added for commit yet.
 - Git tracks the changes, but they are not ready to save in history.
-

2. Staging Area

- The staging area contains files that are ready to be committed.
- Files move to the staging area using the `git add` command.

```
git add file-name
```